



MCDI500

Evaluación Sumativa 2 — Fase 2

Notebook de obtención, limpieza y transformación de datos

Título del proyecto:

Factores socioeconómicos y de preparación previa asociados al rendimiento académico

Integrantes: *Juan de Dios Díaz Ríos*

Francisco Fariña Molina

Constanza Moreno Giacometto

Yenne Sepúlveda Jerez

Curso: *MCDI500 Programación para la Ciencia de Datos*

Docente: *Omar Salinas Silva*

Fecha: *13/06/2026*

Índice

I. Introducción	3
II. Diseño de soluciones algorítmicas eficientes	5
Codificación funcional y arquitectura básica del script.....	5
Preprocesamiento y transformación del dataset	8
Validación técnica y verificación del código	11
Preprocesamiento y transformación de datos.....	13
III. Notebooks ejecutables (F2).....	16
IV. Repositorio GitHub (F2).....	17
V. Bibliografía (APA 7)	21

I. Introducción

El presente informe corresponde a la Fase 2 ABP¹ del proyecto transversal de la asignatura Programación para la Ciencia de Datos del Magíster en Ciencia de Datos e Inteligencia Artificial impartido por la Universidad Andrés Bello. El proyecto se desarrolla en el ámbito educativo, un área en la que el rendimiento académico constituye una temática de gran interés debido a su impacto en los procesos de aprendizaje y en la toma de decisiones basadas en evidencia. En este contexto, se seleccionó el conjunto de datos Student Performance Dataset, desarrollado por Cortez y Silva (2008), disponible en UCI Machine Learning Repository (Cortez, 2008) a través de Dua y Graff (2019), por la amplitud y diversidad de información que proporciona sobre variables individuales, familiares, socioeconómicas, hábitos de estudio y antecedentes académicos asociados al desempeño de los estudiantes.

El dataset contiene información de dos establecimientos educacionales portugueses: Gabriel Pereira (GP) y Mousinho da Silveira (MS), e incluye registros de 649 estudiantes de la asignatura de Portugués y 395 estudiantes de Matemáticas, cada uno descrito mediante 33 variables que abarcan factores directos del rendimiento (studytime, failures, absences), contexto familiar (Medu, Fedu, famrel), comportamiento social (goout, Dalc, Walc) y perfil sociodemográfico (sex, age, school, address). Asimismo, incorpora dos calificaciones parciales y una calificación final. Esta última, correspondiente a la variable G3 (escala de 0 a 20), constituye la variable objetivo central del estudio, cuya comprensión y eventual predicción representan el principal propósito analítico del proyecto

La problemática abordada en este proyecto consiste en comprender qué factores socioeconómicos y de preparación previa podrían explicar las diferencias en el rendimiento académico entre estudiantes de dos establecimientos educacionales. La riqueza y cantidad de variables disponibles ofrecen una oportunidad para estudiar el fenómeno desde una perspectiva integral e identificar patrones y relaciones entre distintos factores que influyen en los resultados de aprendizaje.

Sobre la base de los objetivos definidos en la Fase 1, durante la Fase 2 se desarrollaron las actividades de exploración y limpieza de datos, orientadas a caracterizar las variables del conjunto de datos, identificar patrones y preparar la información para las etapas posteriores de análisis y modelado, que se desarrollará en las siguientes asignaturas del Magister.

Objetivo general

Identificar y analizar los factores socioeconómicos y de preparación previa que explican las diferencias en el rendimiento académico entre estudiantes de dos establecimientos educacionales, mediante técnicas de análisis exploratorio y modelado predictivo sobre el Student Performance Dataset (Cortez y Silva, 2008).

¹ ABP: El Aprendizaje Basado en Proyectos (ABP) es una estrategia de enseñanza y aprendizaje centrada en el estudiante, en la que se construye el conocimiento a través de la planificación, desarrollo y presentación de proyectos, favoreciendo el trabajo colaborativo, la investigación y la resolución de problemas (Ruiz Hidalgo & Ortega-Sánchez, 2022).

Objetivos específicos

Se definen a continuación los objetivos específicos del proyecto (Tabla 1):

N°	Objetivo específico	Fase
OE1	Configurar un entorno de trabajo reproducible con Python, Jupyter Lab, Git y GitHub, estableciendo la estructura técnica del proyecto conforme a las buenas prácticas del software científico.	F1
OE2	Realizar la carga, inspección y limpieza del Student Performance Dataset, verificando la integridad, calidad y estructura de los datos para garantizar su adecuación al análisis posterior.	F1 – F2
OE3	Ejecutar un análisis exploratorio de datos (EDA) que permita identificar distribuciones, correlaciones y patrones iniciales entre las variables socioeconómicas y el rendimiento académico.	F2
OE4	Desarrollar y evaluar modelos predictivos —regresión lineal múltiple y árboles de decisión— para identificar los factores de mayor incidencia en la calificación final G3.	F3
OE5	Documentar y comunicar los hallazgos del proyecto mediante notebooks reproducibles, un repositorio GitHub estructurado y un informe técnico formal.	F1, F2, F3, F4

Tabla 1. *Objetivos específicos*

En concordancia con los objetivos específicos definidos para el proyecto, el presente informe se centra en las actividades correspondientes a la Fase 2. En particular, aborda la continuación del Objetivo Específico 2 (OE2), asociado a la carga, inspección y limpieza del Student Performance Dataset, y desarrolla el Objetivo Específico 3 (OE3), orientado a la ejecución del análisis exploratorio de datos (EDA) con el fin de identificar distribuciones, correlaciones y patrones iniciales entre las variables socioeconómicas y el rendimiento académico.

Durante la Fase 2: Exploración y limpieza de datos (EDA), se trabajó con ambos conjuntos de datos por separado (student-mat.csv y student-por.csv), aplicando una estrategia de limpieza, preprocesamiento y transformación que comenzó con la obtención de estadísticas descriptivas para caracterizar la distribución de las variables numéricas y categóricas. Posteriormente, se realizó un análisis de correlaciones con el propósito de identificar asociaciones entre factores de estudio, antecedentes familiares, hábitos sociales y las calificaciones obtenidas. Asimismo, se efectuó la detección de valores atípicos en variables como absences, G1 y G2, evaluando su impacto y aplicando winsorización cuando correspondió.

Considerando que el conjunto de datos incorpora variables categóricas binarias y nominales, se aplicaron técnicas de codificación (binaria y One-Hot Encoding) para transformarlas a representaciones numéricas compatibles con los procesos de análisis y modelado de la Fase 3 que se realizará posteriormente en la siguiente asignatura del Magister. Adicionalmente, se generaron variables derivadas (aprobado, progreso_g1_g3, nivel_alcohol, edu_familiar_media) que sintetizan información relevante para el análisis.

Como resultado de esta fase, se obtuvo un conjunto de datos limpio, consistente y adecuadamente transformado, el cual fue exportado a la carpeta data/processed/ del repositorio. Asimismo, se logró una comprensión más profunda de las características del dataset y de las relaciones existentes entre los factores socioeconómicos, familiares y académicos que podrían influir en el rendimiento de los estudiantes. Lo anterior proporciona una base sólida, trazable y reproducible para las etapas posteriores de análisis y modelado correspondientes a la Fase 3 (F3).

Los procedimientos definidos para la exploración, limpieza, preprocesamiento y transformación de los datos fortalecen la reproducibilidad, la trazabilidad y la consistencia metodológica del pipeline de datos, apoyados en el entorno reproducible basado en Python, Jupyter Lab, Git y GitHub configurado en la Fase 1. Cada actividad se ejecutó mediante código documentado y organizado en el notebook **F2_limpieza.ipynb**, permitiendo replicar los resultados bajo las mismas condiciones. Las decisiones técnicas adoptadas como winsorización, codificación de variables categóricas, creación de variables derivadas quedan registradas y versionadas mediante Git y GitHub, asegurando la trazabilidad de los cambios (Salinas, 2026a; Universidad Andrés Bello, 2026).

II. Diseño de soluciones algorítmicas eficientes

Codificación funcional y arquitectura básica del script

Como parte de la arquitectura básica del script, se define cada etapa del proceso de preparación de datos e implementación mediante funciones independientes con responsabilidades definidas. Esta organización permite mejorar la reutilización del código, simplificar su mantenimiento y facilitar futuras modificaciones o ampliaciones del pipeline.

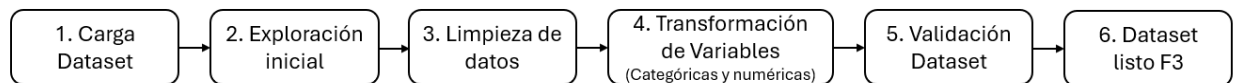


Imagen 1. Etapas de la Fase 2

El notebook F2_limpieza.ipynb implementa una arquitectura modular basada en funciones con responsabilidad única, siguiendo las buenas prácticas de programación en Python (Salinas, 2026c), importadas desde el módulo src/functions.py establecido en la Fase 1. Cada etapa del pipeline —carga, exploración, limpieza, transformación y validación se encapsula en una función independiente con parámetros claramente definidos y salidas verificables, garantizando un flujo de datos coherente y reutilizable entre ambos datasets (Matemáticas y Portugués).

Como decisión técnica, y en concordancia con el criterio adoptado durante la Fase 1, se optó por mantener ambas fuentes de datos en archivos independientes y cargarlas por separado. Esta decisión tuvo como propósito preservar la estructura original de cada conjunto de datos, garantizando la trazabilidad de la información y la reproducibilidad del proceso de trabajo.

En la presente Fase 2 se mantuvo la misma premisa metodológica, con el fin de seguir una secuencia consistente con las etapas definidas para la asignatura de Ciencia de Datos. No obstante, esta decisión no excluye la posibilidad de realizar posteriormente una integración de ambas fuentes y repetir las etapas de análisis bajo un conjunto de datos unificado.

Asimismo, mantener los archivos por separado permitió evitar transformaciones prematuras que pudieran introducir errores, inconsistencias o sesgos en la información. Si bien la unificación de las fuentes no fue abordada en su totalidad durante esta fase, se han incorporado en la sección de referencias y anexos algunas consideraciones y pasos iniciales que servirán de base para una eventual integración en etapas futuras del proyecto.

La Tabla 2 describe las funciones efectivamente utilizadas en el pipeline ejecutado, su propósito y la evidencia de ejecución obtenida.

Función	Etapas del pipeline	Propósito y evidencia de ejecución
<i>cargar_datos()</i>	1. Carga Dataset	Lee student-mat.csv y student-por.csv. Resultado: 395×33 y 649×33 filas/columnas respectivamente. Salidas: df_mat y df_por
<i>mostrar_primeras_filas(df, n)</i>	2. Exploración inicial	Muestra las primeras n filas de cada dataset para inspección visual inicial.
<i>resumen_dataset(df)</i>	2. Exploración inicial	Reporta filas, columnas, duplicados, nulos y tipos de datos. Resultado: 0 duplicados y 0 nulos en ambos datasets. Además de lo anterior brinda información de los tipos de datos de las columnas.
<i>df.describe().round(2)</i>	2. Exploración inicial	Estadística descriptiva. Genera resumen estadístico de todas las variables numéricas. Cantidad de registros, media, desviación estándar, mínimo, cuartiles y máximo valor.
<i>validar_rangos(df, nombre)</i>	2. Exploración inicial	Verifica que cada variable numérica esté dentro de su rango teórico (p. ej. $G1, G2, G3 \in [0, 20]$). Resultado: 0 valores fuera de rango en las 16 variables de ambos datasets.
<i>detectar_outliers_iqr(df, vars, nombre)</i>	2. Exploración inicial	Aplica el criterio de Tukey ($Q1 - 1.5 \cdot IQR$, $Q3 + 1.5 \cdot IQR$) (Tukey, 1977), sobre age, absences, G1, G2, G3. Detecta 15 outliers en absences (MAT) y 21 (POR).

<i>analizar_correlaciones(df, nombre, guardar_figura)</i>	2. Exploración inicial	Calcula la correlación de Pearson de las 16 variables numéricas con G3 y genera mapa de calor. Resultado principal: G2 ($r=0.90$ MAT / 0.92 POR) y G1 ($r=0.80$ / 0.83).
<i>analizar_g3_cero(df, nombre)</i>	2. Exploración inicial	Desagrega los casos $G3=0$ en deserción ($G2=0 \wedge G3=0$) y reprobación ($G2>0 \wedge G3=0$). Resultado: 38 casos MAT (13+25) y 15 casos POR (7+8).
<i>limpiar_dataset(df,asignatura)</i>	3. Limpieza de datos	Elimina duplicados (0 en ambos), aplica strip a 17 columnas de texto, crea flags 'desercion' y 'asignatura'.
<i>winsorizacion(df, col, percentil)</i>	3. Limpieza de datos	Aplica capping al percentil 95 sobre absences, preservando el valor original en absences_original. Resultado: 20 valores ajustados (MAT, cap=18) y 32 (POR, cap=12). Salidas: df_mat_clean y df_por_clean
<i>codificar_binarias(df)</i>	4. Transformación de variables	Convierte 8 variables yes/no a 1/0 y genera 5 dummies adicionales (sex_F, address_U, famsize_GT3, Pstatus_T, school_GP).
<i>codificar_ohe(df)</i>	4. Transformación de variables	Aplica One-Hot Encoding a Mjob, Fjob, reason y guardian, generando 13 columnas dummy ($4+4+3+2$) y eliminando las 4 columnas originales.
<i>crear_variables_derivadas(df)</i>	4. Transformación de variables	Genera aprobado, progreso_g1_g3, nivel_alcohol y edu_familiar_media (4 columnas nuevas).
<i>validar_dataset_final (df_orig, df_proc, nombre)</i>	5. Validación Dataset	Ejecuta asserts sobre filas conservadas, nulos, rangos de G1/G2/G3, y coherencia de los flags 'aprobado' y 'desercion'. Resultado: VALIDACIÓN EXITOSA en ambos datasets. Como salida se genera un resumen del pipeline.
<i>integrar_datasets(df_mat, df_por)</i>	Anexo. Integración	Concatena verticalmente ambos datasets transformados, agregando la columna 'asignatura'. Resultado: 1044 filas (395 mat + 649 por) × 54 columnas.
<i>exportar_dataset(df, ruta, descripcion)</i>	Transversal etapas	Exporta cada DataFrame a data/processed/ con reporte de dimensiones y tamaño en KB. Utilizada en 5 puntos del pipeline.

Tabla 2: Codificación funcional utilizadas

El flujo de datos sigue la secuencia: cargar_datos() → resumen_dataset() / validar_rangos() / detectar_outliers_iqr() / analizar_correlaciones() / analizar_g3_cero() → limpiar_dataset() + winsorizacion() → codificar_binarias() + codificar_ohe() + crear_variables_derivadas() → validar_dataset_final() → exportar_dataset(). Esta secuencia fue ejecutada de forma continua

(Restart & Run All), con numeración de ejecución correlativa de 1 a 24, confirmando la coherencia entre entradas, procesamiento y salidas en cada etapa.

Preprocesamiento y transformación del dataset

El preprocesamiento del Student Performance Dataset siguió una secuencia de diagnóstico, limpieza y transformación fundamentada en las recomendaciones de García et al. (2015) y en la documentación oficial de Pandas (The pandas development team, 2024) y NumPy (NumPy Developers, 2024; Salinas, 2026b).

Diagnóstico inicial

La exploración inicial mediante `resumen_dataset()` confirmó que ambos datasets no presentan valores nulos ni filas duplicadas (Tabla 3). La función `validar_rangos()` verificó que las 16 variables numéricas de cada dataset se encuentran dentro de sus rangos teóricos definidos por Cortez y Silva (2008), sin excepciones.

Indicador	Matemáticas (395×33)	Portugués (649×33)
Valores nulos	0	0
Filas duplicadas	0	0
Variables fuera de rango	0 de 16	0 de 16
Outliers en absences (IQR)	15 (3.8%)	21 (3.2%)
G3 = 0 (total)	38 (13 deserción + 25 reprobados)	15 (7 deserción + 8 reprobados)
G2 mean / G1 mean	10.71 / 10.91	11.57 / 11.40

Tabla 3. Diagnóstico inicial

Estrategia de limpieza

La Tabla 4 presenta las decisiones de limpieza adoptadas y su justificación técnica, conforme a lo ejecutado en la celda 28 del notebook.

Caso identificado	Variable(s)	Decisión técnica	Justificación
Sin valores nulos / duplicados	Todas	No se requiere imputación ni eliminación de filas	Confirmado por <code>resumen_dataset()</code> : 0 nulos y 0 duplicados en ambos datasets (Cortez & Silva, 2008)
Texto en variables categóricas	17 columnas object	<code>strip()</code>	Estandariza espacios para evitar inconsistencias en codificación posterior

Estudiantes con G3=0	G3, G2	Conservar todos los registros; crear flag desercion	38 (MAT) y 15 (POR) casos son valores válidos en [0,20]; eliminarlos sesgaría la muestra excluyendo a los estudiantes con peor desempeño
Outliers en absences (IQR)	absences	Winsorización al percentil 95	15/21 valores extremos (hasta 75 ausencias) están dentro del rango válido [0,93] — comportamiento real, no error. Se preserva absences_original para trazabilidad

Tabla 4. Decisiones de limpieza

Tratamiento de outliers en columna absences, opciones consideradas

Antes de aplicar un tratamiento, se evaluaron seis estrategias alternativas para el manejo de los valores extremos detectados en absences (Tabla 5).

Opción	En qué consiste	Efecto sobre el caso de 75 ausencias	Motivo de descarte / adopción
Eliminar la fila	Borrar el registro completo	La fila desaparece (395→394)	Pierde las 32 demás variables del estudiante, incl. G1/G2/G3
Eliminar solo el valor (NaN)	Vaciar absences, conservar la fila	absences = NaN	Introduce un nulo donde el diagnóstico confirmó 0; traslada el problema a una imputación posterior
Imputar media/mediana	Reemplazar por el promedio/mediana	absences = 4 en vez de 75	Sustituye un comportamiento real por un dato ficticio
Transformación logarítmica	$\log(\text{absences}+1)$ a toda la columna	$75 \rightarrow \log(76) \approx 4.33$	Altera la escala de toda la variable, dificulta interpretación
Recorte (trimming)	Eliminar filas sobre un umbral	15/21 filas eliminadas	Mismo problema que eliminar filas
Winsorización p95 (adoptada)	Cap al valor del percentil 95	$75 \rightarrow 18$ (cap p95 MAT)	Conserva la fila completa, atenúa el extremo sin inventar datos; absences_original preserva trazabilidad

Tabla 5. Tratamiento outliers

Transformaciones aplicadas

La Tabla 6 detalla las transformaciones aplicadas y su efecto neto sobre la estructura del dataset (verificado: 36→54 columnas, +18 netas).

Transformación	Columnas eliminadas	Columnas nuevas	Justificación técnica
Codificación binaria yes/no	— (en su lugar)	8 columnas (mismo nombre, valores 1/0)	schoolsup, famsup, paid, activities, nursery, higher, internet, romantic → permiten correlación y modelado
Dummies binarias propias	0	5: sex_F, address_U, famsize_GT3, Pstatus_T, school_GP	Variables nominales de 2 categorías; se preserva la columna original junto a su versión numérica
One-Hot Encoding	4: Mjob, Fjob, reason, guardian	13: Mjob_health/other/services/teacher (4), Fjob_idem (4), reason_home/other/reputation (3), guardian_mother/other (2)	Variables nominales >2 categorías sin orden natural; OHE evita sesgo ordinal artificial
Variables derivadas	0	4: aprobado, progreso_g1_g3, nivel_alcohol, edu_familiar_media	aprobado=1 si $G3 \geq 10$; progreso_g1_g3= $G3 - G1$; nivel_alcohol= $(Dalc + Walc)/2$; edu_familiar_media= $(Medu + Fedu)/2$
TOTAL	4 eliminadas	22 nuevas → neto +18	36 columnas (post-limpieza) → 54 columnas (post-transformación)

Tabla 6. Transformaciones aplicadas

Validación técnica y verificación del código

La validación del pipeline se realizó mediante la función `validar_dataset_final()`, que ejecuta comprobaciones tipo `assert` sobre el dataset procesado, comparándolo contra el dataset original. La Tabla 7 resume las validaciones aplicadas y su resultado.

Validación	Tipo de caso	Resultado MAT	Resultado POR
Filas conservadas	Caso normal	395	649
Sin nulos en columnas originales	Caso normal	0	0
G1, G2, G3 intactos	Caso normal		
Flag 'aprobado' coherente ($G3 \geq 10$)	Caso normal	265 aprobados	549 aprobados
Flag 'desercion' coherente	Caso límite	13 casos	7 casos
Sin filas duplicadas	Caso normal	0	0
G1, G2, G3 en rango [0,20]	Caso límite	✓	✓
Resultado global	—	Validación exitosa	Validación exitosa

Tabla 7. Validación técnica

Adicionalmente, la celda 36 documenta evidencia de calidad mediante conteos antes/después (filas, columnas, nulos, duplicados) para ambos datasets, confirmando que el pipeline no introdujo inconsistencias: 0 nulos y 0 duplicados se mantienen estables a lo largo de todo el proceso.

Análisis de correlaciones con la variable objetivo (G3)

El análisis de correlación de Pearson sobre las 16 variables numéricas (celdas 18 y 20) constituye evidencia de validación adicional, al confirmar relaciones esperadas según la literatura (Cortez & Silva, 2008). La Tabla 8 presenta las correlaciones más relevantes con G3 para ambos datasets.

Variable	Corr. con G3 — MAT	Corr. con G3 — POR	Interpretación
G2	0.905	0.919	Correlación más fuerte en ambos datasets — el rendimiento del segundo período predice fuertemente la nota final
G1	0.801	0.826	Segunda correlación más fuerte; confirma estabilidad del rendimiento durante el año
failures	-0.360	-0.393	Correlación negativa más fuerte entre las variables socioeconómicas/de contexto — cada reprobación previa reduce notablemente G3
Medu	0.217	0.240	Educación de la madre — correlación positiva moderada, consistente en ambas asignaturas
studytime	0.098	0.250	Notablemente más relevante en Portugués ($r=0.25$) que en Matemáticas ($r=0.10$) — posible diferencia entre asignaturas a profundizar en F3
goout	-0.133	-0.088	Salidas con amigos — correlación negativa débil, algo más marcada en Matemáticas
absences	0.034	-0.091	Correlación cercana a cero en ambos casos — el efecto de las ausencias sobre G3 no es lineal o se manifiesta indirectamente

Tabla 8. Análisis de correlaciones de variables

Validación técnica y verificación del código

La validación técnica y verificación del código se realizó mediante pruebas controladas que permitieron comprobar el correcto funcionamiento del pipeline de datos en distintos escenarios. Para ello, se evaluaron casos normales, casos límite y posibles excepciones, verificando que las funciones respondieran de manera consistente ante entradas válidas, datos vacíos, columnas inexistentes o valores faltantes. Además, los resultados intermedios fueron documentados mediante salidas visibles en Jupyter Lab, tales como capturas de ejecución, logs, tablas resumen y celdas de validación. Este proceso permitió asegurar la trazabilidad de cada verificación realizada y confirmar que el dataset final conserva coherencia, integridad y consistencia antes de ser utilizado en la Fase 3.

La Tabla 9 presenta una ruta de validaciones efectuadas e implementadas en el notebook de la fase 2.

Variable	Interpretación
1. Configurar logs para trazabilidad	El notebook incorpora mensajes impresos durante la ejecución del proceso, tales como carga exitosa de datasets, validaciones, transformaciones, exportaciones y resumen ejecutivo. Estos mensajes permiten seguir la trazabilidad del flujo ejecutado.
2. Verificación de caso normal	Se validó la carga correcta de los archivos student-mat.csv y student-por.csv, verificando sus dimensiones, primeras filas, tipos de datos, valores nulos y duplicados mediante funciones como cargar_dataset(), mostrar_primeras_filas() y resumen_dataset().
3. Verificación de casos límite	Se revisaron rangos válidos de variables numéricas mediante validar_rangos(), además de analizar casos especiales como valores extremos en absences y registros con G3 = 0.
4. Verificación de excepciones	La carga y validación de los datasets se realizó mediante funciones externas, lo que permitió controlar errores asociados a rutas, estructura de archivos o datos fuera de rango antes de continuar con el pipeline.
5. Comprobación de resultados intermedios	Se revisaron salidas parciales del proceso, como estadísticas descriptivas, distribuciones categóricas, análisis de outliers, correlaciones, tratamiento de absences y creación de variables derivadas.
6. Prueba manual del pipeline	Se ejecutó manualmente la secuencia completa del pipeline: carga, inspección, limpieza, winsorización, codificación de variables, creación de variables derivadas y exportación de archivos procesados.
7. Validación final del dataset procesado	Se aplicó la función validar_dataset_final() para comparar los datasets originales con los datasets procesados, revisando filas, columnas, nulos, duplicados y consistencia general posterior al procesamiento.
8. Guardar evidencia de validación	Se guardaron archivos procesados en data/processed/ y visualizaciones en data/development/, además de imprimir un resumen ejecutivo con los principales resultados de calidad, limpieza y transformación de datos.

Tabla 9. Validación técnica

La validación del pipeline se realizó mediante la función validar_dataset_final(), que ejecuta comprobaciones tipo assert sobre el dataset procesado, comparándolo contra el dataset original.

La Tabla 10 resume las validaciones aplicadas y su resultado.

Validación	Tipo de caso	Resultado MAT	Resultado POR
Filas conservadas	Caso normal	395 ✓	649 ✓
Sin nulos en columnas originales	Caso normal	0 ✓	0 ✓
G1, G2, G3 intactos	Caso normal	✓	✓
Flag 'aprobado' coherente (G3≥10)	Caso normal	265 aprobados ✓	549 aprobados ✓
Flag 'desercion' coherente	Caso límite	13 casos ✓	7 casos ✓
Sin filas duplicadas	Caso normal	0 ✓	0 ✓
G1, G2, G3 en rango [0,20]	Caso límite	✓	✓
Resultado	—	Validación exitosa	Validación exitosa

Tabla 10. Validaciones aplicadas y resultado.

Adicionalmente, la celda 36 documenta evidencia de calidad mediante conteos antes/después (filas, columnas, nulos, duplicados) para ambos datasets, confirmando que el pipeline no introdujo inconsistencias: 0 nulos y 0 duplicados se mantienen estables a lo largo de todo el proceso.

Preprocesamiento y transformación de datos

El preprocesamiento y transformación de datos se realizó antes del análisis y modelado con el objetivo de asegurar que el dataset fuera coherente, consistente y apto para su uso en la Fase 3. Para ello, se ejecutaron operaciones de limpieza, eliminación de duplicados, revisión de valores faltantes, conversión de tipos de datos, codificación de variables categóricas y normalización de variables numéricas. Cada transformación quedó documentada mediante comentarios, salidas intermedias y archivos de validación, permitiendo justificar técnicamente su aplicación y mantener la trazabilidad del proceso.

El preprocesamiento y transformación del dataset se realizó mediante un flujo controlado que consideró la revisión exhaustiva de valores faltantes, duplicados, tipos de datos e inconsistencias estructurales. Posteriormente, se aplicaron transformaciones adecuadas sobre las variables, incluyendo casting de tipos, codificación de variables categóricas y normalización de variables numéricas. Estas acciones permitieron asegurar la coherencia, integridad y consistencia del dataset resultante (ver Tabla 11).

Función	Etapas del pipeline
1. Carga Dataset	<code>cargar_datos()</code>
2. Eliminar duplicados	<code>limpiar_dataset()</code> , <code>duplicated()</code>
3. Revisar valores faltantes	<code>resumen_dataset()</code> , <code>isnull()</code> , <code>sum()</code>
4. Imputar valores faltantes si existen	No se aplicó imputación, ya que ambos datasets no presentaron valores faltantes. La validación se realizó mediante <code>resumen_dataset()</code> e <code>isnull().sum()</code> .
5. Conversión de tipos de datos	
6. Codificación de variables categóricas	<code>codificar_binarias()</code> , <code>codificar_ohe()</code>
7. Normalización de variables numéricas	<code>winsorizacion()</code>
8. Validar consistencia del dataset	<code>validar_rangos()</code> , <code>validar_dataset_final()</code> , <code>analizar_g3_cero()</code> , <code>analizar_correlaciones()</code>
Validaciones finales	
9. Documentar operaciones realizadas	<code>print()</code> , <code>mostrar_primeras_filas()</code> , <code>resumen_dataset()</code> , <code>describe()</code> , generación de gráficos y resumen ejecutivo del proceso
Creación del diccionario	
10. Guardar evidencia del preprocesamiento	<code>savefig()</code> , <code>makedirs()</code> , almacenamiento de gráficos en <code>data/development/</code> , generados con <code>matplotlib</code> (Hunter, 2007) y <code>seaborn</code> (Waskom, 2021)

Tabla 11. Preprocesamiento

Las transformaciones aplicadas dejaron el dataset listo para las etapas de análisis exploratorio y modelado predictivo de la Fase 3. La Tabla 12 resume la estructura final de ambos datasets exportados a `data/processed/`.

Característica	<code>student_mat_clean_encode.csv</code>	<code>student_por_clean_encode.csv</code>
Filas	395	649
Columnas (original → final)	33 → 54	33 → 54
Valores nulos	0	0
Duplicados	0	0
Variables codificadas binarias	8 (en su lugar) + 5 dummies nuevas	Misma estructura
Variables OHE	13 (Mjob, Fjob, reason, guardian)	Misma estructura
Variables derivadas	aprobado, progreso_g1_g3, nivel_alcohol, edu_familiar_media	Misma estructura

Aprobados ($G3 \geq 10$)	265 (67.1%)	549 (84.6%)
Tamaño archivo	48.5 KB	77.7 KB

Tabla 12. Estructura final datasets

III. Notebooks ejecutables (F2)

El notebook de la Fase 2 fue estructurado siguiendo un flujo lógico y coherente que refleja todas las etapas del pipeline de datos (ver Tabla 13):

Sección	Contenido	Criterio rúbrica
1	Carga del dataset: cargar_datos() para MAT y POR	Codificación funcional
2	Exploración inicial: primeras filas, resumen, descriptivos, validación de rangos, distribución de categóricas, outliers IQR, correlaciones, análisis G3=0, histogramas	Preprocesamiento y transformación
3	Limpieza de datos: limpiar_dataset() + winsorizacion(), exportación de datasets _clean	Preprocesamiento y transformación
4	Transformación de variables: codificar_binarias(), codificar_ohe(), crear_variables_derivadas(), exportación _clean_encode	Preprocesamiento y transformación
5	Validación Dataset: validar_dataset_final(), evidencia de conteos antes/después	Validación técnica
6	Resumen del pipeline: resumen ejecutivo con hallazgos, transformaciones y archivos exportados	Notebook documentado
7	Referencias (APA 7)	Aspectos formales
8	Anexo — Consolidación de datasets: integrar_datasets(), exportación del dataset combinado	Notebook documentado

Tabla 13. Estructura del Notebook del proyecto.

Cada etapa está precedida por una celda Markdown que explica:

- Qué operación se realiza en esa sección.
- Por qué se realiza la transformación o verificación.
- Resumen hallazgos y/o conclusiones de la operación sobre el dataset.

El notebook fue ejecutado siguiendo la práctica de Restart & Run All, confirmando que todas las celdas se ejecutan de manera consecutiva y sin errores de inicio a fin. La numeración de ejecución se mantiene continua, lo que garantiza trazabilidad y reproducibilidad del pipeline de datos.

Además, se incorporaron comentarios dentro del código para justificar decisiones técnicas y asegurar que cualquier desarrollador que revise el notebook pueda comprender el flujo de trabajo y las transformaciones aplicadas. La documentación y las salidas visibles del notebook constituyen evidencia completa de que el pipeline funciona correctamente y de manera consistente con los objetivos de la Fase 2 del proyecto.

IV. Repositorio GitHub (F2)

El repositorio del proyecto fue organizado siguiendo una estructura clara y orientada a facilitar el desarrollo colaborativo y la reproducibilidad del pipeline de procesamiento de datos. Para la Fase 2 se incorporó una carpeta específica que contiene todos los elementos relacionados con el preprocesamiento y transformación del dataset, manteniendo una separación lógica entre código, datos y documentación.

La estructura del proyecto incluye directorios destinados a los notebooks de trabajo, los datos procesados y el código fuente reutilizable, permitiendo una navegación sencilla y una adecuada organización de los archivos. Asimismo, se conserva el dataset original separado de las versiones transformadas, evitando modificaciones sobre los datos fuente y favoreciendo la trazabilidad del proceso.

Durante el desarrollo se realizaron commits periódicos con mensajes descriptivos que reflejan el avance de cada etapa del pipeline, incluyendo la carga del dataset, la exploración inicial, la limpieza de datos, la codificación de variables y la exportación del conjunto de datos procesado. Esta estrategia permite reconstruir la evolución del proyecto y facilita la identificación de los cambios realizados por cada integrante del equipo.

El archivo README.md fue actualizado para documentar la organización del repositorio, las dependencias necesarias para ejecutar el proyecto y las instrucciones para reproducir el notebook correspondiente a la Fase 2. Además, se describe la función de cada carpeta y el flujo general del pipeline, permitiendo que cualquier usuario pueda comprender y ejecutar el proyecto sin información adicional.

La correspondencia entre el informe, el notebook y el repositorio se mantiene de forma explícita. Las funciones implementadas, las transformaciones aplicadas y los resultados obtenidos en el notebook están respaldados por los archivos almacenados en GitHub, garantizando la trazabilidad completa del proceso de preparación de datos y facilitando futuras tareas de mantenimiento o ampliación del proyecto.

El repositorio del proyecto en GitHub fue actualizado para reflejar el avance de la Fase 2, incorporando la estructura de carpetas correspondiente, los datasets procesados y la documentación técnica actualizada. URL del repositorio: <https://github.com/ffarina11/proyecto-grupo7-mcdi500>. En la tabla siguiente (Tabla 14) se detalla el contenido del repositorio.

Carpeta / Archivo	Contenido F2
notebooks/F2_limpieza.ipynb	Notebook principal de la Fase 2: pipeline completo ejecutado (24 celdas, #1–#24)
data/raw/	student-mat.csv y student-por.csv (sin modificaciones desde Fase 1)
data/processed/	student_mat_clean.csv, student_por_clean.csv (395×36 / 649×36), student_mat_clean_encode.csv, student_por_clean_encode.csv (395×54 / 649×54)
data/development/	Figuras generadas: distribución de categóricas, boxplot absences, correlacion_matematicas.png, correlacion_portugues.png, histograma G3

src/	functions.py y librerias.py — funciones del pipeline F2 (cargar_datos, limpiar_dataset, winsorizacion, codificar_binarias, codificar_ohe, crear_variables_derivadas, validar_dataset_final, integrar_datasets, exportar_dataset)
README.md	Actualizado con instrucciones de ejecución del notebook F2, dependencias y organización del proyecto
requirements.txt	Sin cambios desde Fase 1 (pandas, numpy, matplotlib, seaborn, scikit-learn (Pedregosa et al., 2011), jupyterlab)

Tabla 14: Detalle repositorio.

En la imagen 2 se visualizan el directorio del proyecto, repositorio GitHub:

```

proyecto-grupo7-mcdi500/
├── data/
│   ├── raw/                    # Datos originales sin modificar (.csv)
│   │   ├── student-mat.csv
│   │   └── student-por.csv
│   └── processed/              # Datos limpios y transformados (Fase 2)
│       ├── student_mat_clean.csv
│       ├── student_mat_clean_encode.csv
│       ├── student_por_clean.csv
│       ├── student_por_clean_encode.csv
│       ├── student_union_clean_encode.csv
│       └── fig_distribucion_g3.png
├── development/                # Figuras y outputs exploratorios
├── notebooks/
│   ├── F1_Definicion.ipynb     # Fase 1: Definición del problema y EDA inicial
│   └── F2_limpieza.ipynb       # Fase 2: Limpieza, transformación y validación
├── src/                        # Scripts reutilizables
│   ├── functions.py
│   └── librerias.py
├── docs/                      # Documentación complementaria e informes
├── .gitignore                  # Archivos ignorados por Git
├── README.md                   # Descripción general del proyecto
└── requirements.txt            # Dependencias del entorno de desarrollo

```

Imagen 2. Repositorio GitHub Proyecto

Historial de commits de la Fase 2

La Tabla 15 presenta el historial real de commits del repositorio correspondientes a la Fase 2 (12/06/2026), con contribuciones identificables de los 4 integrantes del equipo.

Commit	Autor	Hash	Fecha
F2_carga_data_set_y_exploracion_inicial	Francisco Fariña (ffarina11)	3a44f3e	12/06
F2_cargainicial_exploracion	Francisco Fariña (ffarina11)	0f86ff5	12/06
F2_actualizacion_etapa_limpieza_y_transformacion_variables	Yenne Sepúlveda	2d05cec	12/06
F2_actualizacion_etapa_limpieza_y_transformacion_variables_notebook	Yenne Sepúlveda	30f9353	12/06
F2_validacion_dataset_y_resumen_pipeline_notebook	Constanza Moreno (ConstanzaM0)	6167680	12/06
F2_validacion_dataset_y_resumen_pipeline_notebook	Constanza Moreno (ConstanzaM0)	9903ad5	12/06
F2_Resumen_Pipeline_Referencias_Anexos	Juan de Dios Díaz Ríos	685e654	12/06

Tabla 15. Historial de commits.

El historial muestra una distribución de trabajo por etapas del pipeline: carga y exploración inicial (Francisco Fariña), limpieza y transformación de variables (Yenne Sepúlveda), validación del dataset y resumen del pipeline (Constanza Moreno), y resumen ejecutivo, referencias y anexos (Juan de Dios Díaz Ríos). Adicionalmente, el repositorio registra commits previos (06–09/06/2026) de inicialización de estructura, módulos `src/librerias.py` y `src/functions.py`, y actualización de README, con participación de los 4 integrantes desde el inicio del proyecto.

https://github.com/ffarina11/proyecto-grupo7-mcdi500/commits/main/

ffarina11 / proyecto-grupo7-mcdi500

Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Commits

main

All users All time

Commits on Jun 13, 2026

F2_actualizacion_README_instrucciones_ejecucion_y_estructura ConstanzaM0 committed 6 hours ago	583899a	Copy	<>
--	---------	------	----

Commits on Jun 12, 2026

F2_Resumen_Pipeline_Referencias_Anexos Juan de Dios Diaz Rios committed 18 hours ago	685e654	Copy	<>
F2_validacion_dataset_y_resumen_pipeline_notebook ConstanzaM0 committed 18 hours ago	9903ad5	Copy	<>
F2_validacion_dataset_y_resumen_pipeline_notebook ConstanzaM0 committed 18 hours ago	6167680	Copy	<>
F2_actualizacion_etapa_limpieza_y_transformacion_variables_notebook Yenne Sepulveda committed 19 hours ago	30f9353	Copy	<>
F2_actualizacion_etapa_limpieza_y_transformacion_variables Yenne Sepulveda committed 19 hours ago	2d05cec	Copy	<>
F2_cargainicial_exploracion ffarina11 committed 20 hours ago	0f86ff5	Copy	<>

Imagen 3. Historial de commit (extracto).

V. Bibliografía (APA 7)

- Cortez, P. (2008). *Student Performance* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5TG7T>
- Cortez, P., & Silva, A. (2008). *Using data mining to predict secondary school student performance*. University of Minho. <https://doi.org/10.24432/C5TG7T>
- Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences. <https://archive.ics.uci.edu>
- García, S., Luengo, J., & Herrera, F. (2015). *Data preprocessing in data mining*. Springer.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- NumPy Developers. (2024). *NumPy documentation*. <https://numpy.org/doc/stable/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Ruiz Hidalgo, D., & Ortega-Sánchez, D. (2022). El aprendizaje basado en proyectos: una revisión sistemática de la literatura (2015-2022). *Human Review: International Humanities Review / Revista Internacional de Humanidades*, 14(6), 1-14. <https://doi.org/10.37467/RE VHUMAN.V11.4181>
- Salinas, O. (2026a). *Limpieza y transformación de datos tabulares con Pandas* [Apunte]. Universidad Andrés Bello, Santiago, Chile.
- Salinas, O. (2026b). *NumPy Arrays, Broadcasting y operaciones vectorizadas* [Apunte]. Universidad Andrés Bello, Santiago, Chile.
- Salinas, O. (2026c). *Tipos, variables, booleanos, condicionales, ciclos, funciones, módulos y excepciones en Python* [Apunte]. Universidad Andrés Bello, Santiago, Chile.
- The pandas development team. (2024). *pandas documentation*. <https://pandas.pydata.org/docs/>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- Universidad Andrés Bello. (2026). *Guía de Git y GitHub: Entorno de desarrollo con Python y JupyterLab. Versión para Windows 10 y 11* [Material de apoyo]. Universidad Andrés Bello, Santiago.
- Waskom, M. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>